Tree-Type Network with Neural-Network Nodes

Cats vs Dogs and CIFAR-10: overfitting, double descent, receptive fields and size minimisation

Code (private GitHub repository): GITHUB_LINK_PLACEHOLDER (replace with your repository URL after pushing)

1. Summary

We rebuilt the earlier error-correcting tree so that every node is a small neural network. A single small CNN trunk is trained once per dataset and frozen; each tree node is a tiny MLP head on the trunk features. Trees reach 99.9% (Cats vs Dogs) and 98.5% (CIFAR-10) training accuracy while test accuracy does not improve, which is the requested overfitting regime. We did **not** observe double descent in our sweeps. Gradient saliency suggests root nodes look at the whole object region while some deep nodes have rougher, more instance-specific maps. The smallest network reaching 100% training accuracy has about 20k parameters, and the tree is **not** more parameter-efficient than a single wide head of the same size.

2. Architecture and implementation

Data. Cats vs Dogs: 2,378 images resized to 32x32 (1,902 train / 476 test, stratified). CIFAR-10: 400 images per class (4,000 total; 3,200 train / 800 test), native 32x32. Inputs are scaled to [0,1], no augmentation.

Trunk (shared features). Conv3x3(3-24)-BN-ReLU-MaxPool-Conv3x3(24-48)-BN-ReLU-MaxPool-global average pool-Linear(48-48), 13,584 parameters. It is trained once, end to end, with a pretext head (binary logistic head for Cats vs Dogs, 10-way softmax for CIFAR-10), then frozen and used by every node. BatchNorm was necessary: without it the trunk stalled near 65% train accuracy.

Nodes. Each node is an MLP head Linear(48-h)-ReLU-Linear(h-1) (h=64 for Cats vs Dogs, h=96 for CIFAR in the main runs), trained with Adam on the frozen features.

Tree growing (as in the earlier assignment). After fitting a node, its training samples fall into four groups: C1 (correct, class 0), C2 (correct, class 1), C3 (should be 1, predicted 0) and C4 (should be 0, predicted 1). Child A is trained to output 1 exactly on C3 (C2 is "don't care" and excluded); child B is trained to output 1 exactly on C4 (C1 excluded). The parent's net input becomes net + w_A x_A + w_B x_B with w_A = max over C3 of (1 - net) and w_B = min over C4 of (-1 - net), so every corrected sample crosses the margin. Growth recurses until no errors remain or a depth/node budget is hit. CIFAR-10 uses one tree per class (one-vs-rest) and takes the argmax of the trees' net inputs.

Innovation tried. Multi-level ("hypercolumn") features: heads see global-average-pooled conv1, conv2 and final embedding concatenated, so nodes can use low-level cues. It gave no benefit (Section 3).

3. Main results (overfitting)

Setting	Train acc	Test acc	Nodes	Head params
Cats vs Dogs, root head only	94.7%	77.3%	1	3.2k
Cats vs Dogs, full tree	99.9%	76.7%	19	60.8k
Cats vs Dogs, tree on multi-level features	96.5%	77.1%	18	140.6k
CIFAR-10, root heads only (10 classes)	87.0%	51.8%	10	-
CIFAR-10, full one-vs-rest trees	98.5%	51.1%	70	336.1k

The tree closes most of the remaining train error but does not help test accuracy: the extra nodes memorise training samples. The ten CIFAR trees shared a cap of 60 extra nodes in total (70 nodes including the 10 roots); an earlier uncapped run reached 99.8% train and 53.4% test. The CIFAR trunk itself only reaches about 52% test accuracy,

so the features, not the tree, limit CIFAR generalisation. Test sets are small (476 and 800 images), so differences of 2-4 points are within noise. Single trunk seed.

4. Double descent

We swept the hidden width of a node head (frozen features) with label noise on the training labels, measuring train error on the noisy labels and test error on clean labels. The parameter-count interpolation point is #params = N.

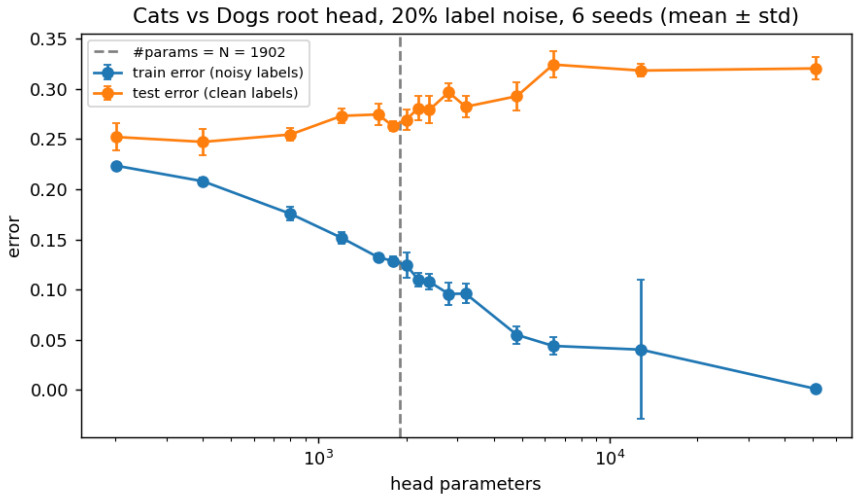


Figure 1. Refined sweep: Cats vs Dogs root head, 20% label noise, 6 seeds, 15 widths (mean +/- std).

Observation. Test error rises from about 25% (width 4) to about 32% (width 128) and then stays flat to width 1024, where train error finally reaches about 0 (51k parameters, roughly 27 times N). There is no second descent. An earlier, coarser sweep (15% noise, 2 seeds; Figure 2, left and middle) showed test error flat at 22-25% on Cats vs Dogs with a small bump at width 32 that is inside the noise, and near-chance, noisy test error (37-46%) for CIFAR cat vs dog (only 160 test images). The right panel of Figure 2 is a whole-tree sweep without label noise: train error falls to 0 at head width 128, test error stays near 22-24%.

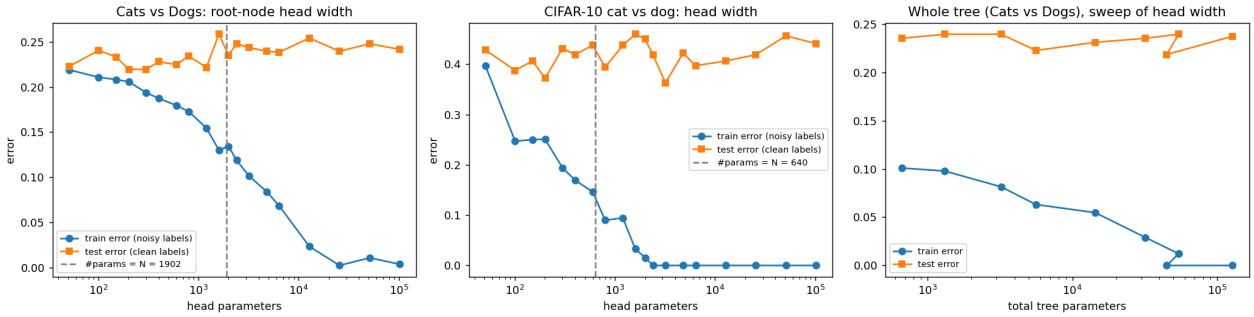


Figure 2. Initial sweeps: root head width (Cats vs Dogs, CIFAR cat vs dog) and whole-tree sweep.

Possible reasons (not tested). (i) The head only sees 48 frozen features, which may cap how much extra width helps; (ii) the sweep stops at width 1024-2048, which may not extend far enough beyond the effective interpolation point; (iii) the noise-induced test-error peak may need other settings (more noise, a different optimiser, or width of the trunk rather than the head). A trunk-width sweep is the natural next experiment.

5. Receptive fields of nodes

For each node we averaged the absolute gradient of its logit with respect to the input pixels over the training images it fires on (up to 150). Theoretical trunk receptive field before pooling is 10x10 pixels, but global average pooling makes every node depend on the whole 32x32 image, so the visualisations show effective rather than architectural fields.

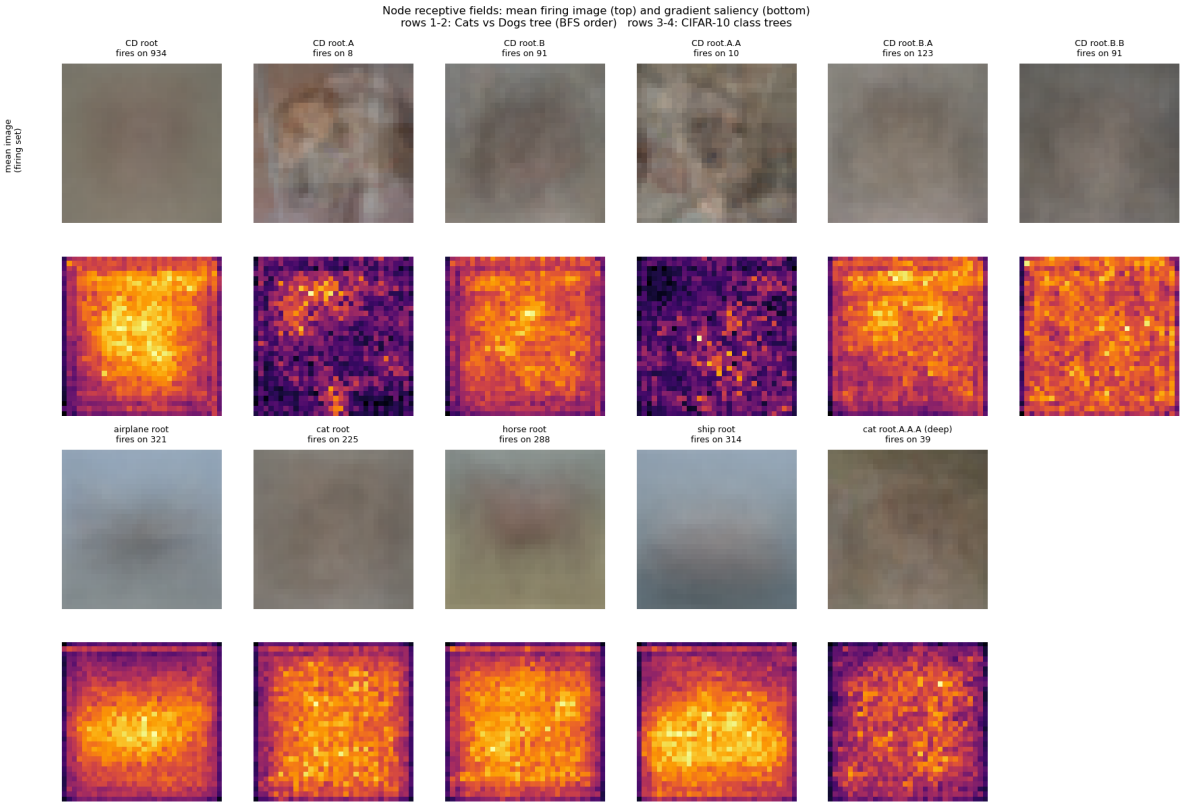


Figure 3. Top rows: Cats vs Dogs tree nodes (breadth-first order). Bottom rows: CIFAR-10 class-tree roots and one deep node. Each node shows its mean firing image and its saliency map.

Observations. Root nodes show smooth, centre-weighted saliency: the airplane and ship roots concentrate on a horizontal band through the middle of the image; the cat and horse roots are more diffuse. Nodes that fire on only 8-10 images look speckled. Because averaging over fewer images is noisier by itself, we ran a control (Table below): speckle is the fraction of saliency energy in a high-pass residual.

Node	Images (n)	Speckle	Root at similar n
root.A	8	0.37	0.43 +/- 0.06 (n=8)
root.A.A	10	0.57	0.38 +/- 0.05 (n=10)
root.B	91	0.21	~0.17 (interpolated, n=40: 0.22, n=150: 0.12)
root.B.A	123	0.22	~0.14 (interpolated)
root.B.B	91	0.46	~0.17 (interpolated)

Interpretation. The roughness of root.A is fully explained by its tiny sample. root.A.A and root.B.B are clearly rougher than the root at comparable sample sizes, consistent with deep nodes responding to image-specific details of the few samples they correct, i.e. memorisation. This is descriptive evidence from gradient saliency on tiny networks, and the root comparisons for n=91 and 123 are interpolated rather than measured. Summary statistics such as saliency entropy (0.99-1.00) and mass in the central region (0.26-0.31) were nearly identical across nodes and are not informative.

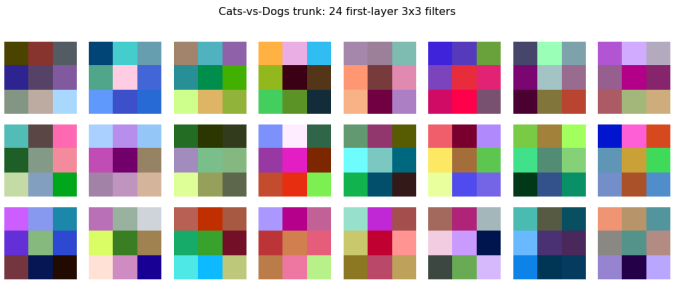


Figure 4. First-layer filters of the Cats vs Dogs trunk.

6. Minimising the network

Our first size grid trained node heads for only 150 epochs, whereas the single-head baseline trained for 1,500, so it was not a fair comparison and no small configuration reached 99% (kept in the repository for transparency). The table below uses a matched budget (1,500 epochs, lr 0.01 per node, depth up to 10, at most 120 extra nodes). Greedy pruning (drop a subtree if training accuracy does not fall) was applied to every configuration that reached at least 99%; it removed nothing.

Trunk	Head h	Nodes	Total params	Train	Test
12/24/24	2	23	4,843	89.5%	74.6%
12/24/24	8	24	8,640	91.6%	75.8%
12/24/24	32	25	24,449	98.0%	72.9%
24/48/48	2	35	17,119	93.2%	76.3%
24/48/48	8	27	24,411	98.0%	77.9%
24/48/48	16	11	22,395	100%	75.8%
24/48/48	32	4	19,988	100%	78.2%
24/48/48	single head, no tree, 32	1	15,185	98.8%	78.2%
24/48/48	single head, no tree, 128	1	19,985	100%	75.6%

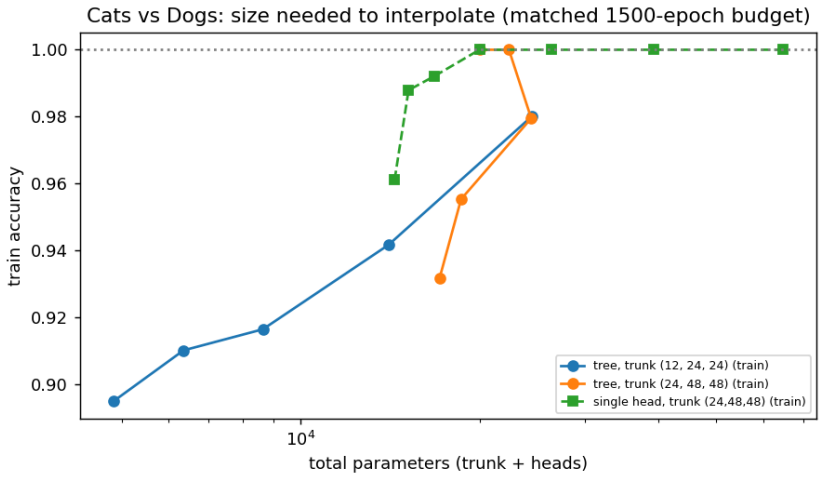


Figure 5. Training accuracy versus total parameters (trunk + heads).

Findings. The smallest network that interpolates the training set has about 20k parameters (13.6k trunk + 6.4k heads). The trunk dominates the count; the smaller 12/24/24 trunk never reached 100% under the matched budget with heads up to width 32 (best 98.0%). With well-trained heads the tree collapses to a few nodes (4 at h=32), and its total size is essentially equal to a single 128-unit head (19,988 vs 19,985 parameters). So in this setting the tree

structure gives no parameter saving for memorisation. Its value would have to lie in interpretability or in cases where one head cannot fit the data. Single seed; test differences of about 2 points are not significant.

7. Interpretations and conclusions

(1) Tree growth makes interpolation easy but not useful: near-100% training accuracy with unchanged test accuracy. (2) The generalisation bottleneck is the trunk representation (about 52% on CIFAR-10 at 32x32 with 13.6k parameters), not the tree. (3) We did not see double descent; with label noise the test error rose and then plateaued. This should be read as "not observed in this setup", not as evidence that it cannot occur here. (4) Deep nodes appear to be more instance-specific than root nodes, supporting the view that the tree stores exceptions to a simple rule. (5) A tree of heads does not beat one wide head in parameter count; matching the optimisation budget was essential to see this, since the unfair first comparison suggested the opposite.

8. Limitations

Images downsampled to 32x32; small subsets; single trunk seed; small test sets; CIFAR trees share a 60-extra-node cap; saliency interpretation is qualitative; first size grid and first double-descent sweep are superseded by the later, better-controlled runs but kept in the repository.

9. Future directions

Sweep trunk width (and depth) jointly with head width to look for model-wise double descent, extending well past the interpolation point and adding more seeds. Train per-branch trunks (fine-tune a copy of the trunk on the data reaching each subtree) so deep nodes can specialise. Use pretrained or larger trunks to lift the CIFAR-10 test ceiling. Prune with a small accuracy tolerance or distil subtrees into the parent. Quantify receptive fields with occlusion tests and matched sample sizes at every node. Extend the regression variant to neural-network nodes.